RunCPM 7xtgv-terminal Build Guide

Table of Contents

1. Getting started
   - Clone
   - Run explicit setup
   - Open a board folder
   - Build, then launch
2. Desktop sessions and cards
   - Repository working card
   - External SD/USB card
   - Session ownership and shutdown
3. Physical hardware
   - Modules and vendor references
   - Build and upload
   - Connection prompts and target guards
   - Mirror without reflashing
   - USB permissions
   - Firmware is not the card
4. Build reference and troubleshooting
   - Quiet output, logs and optional switches
   - Optional optimization selection
   - Prerequisites and manual setup details
   - Artifacts and low-level build entry points
   - Troubleshooting
   - Remaining fresh-machine acceptance

To try the project without installing anything, open the browser application and follow the User Guide.

This guide covers building and running a native desktop on **Linux or Windows**, and building/uploading firmware to physical modules with the Linux tools. No physical board is needed for native desktop use. It runs the integrated RunCPM/terminal and real Arduino_GFX drawing code using your chosen board's display definition. The current terminal is a development testbed for **7xtgv-native**, not the completed native engine.

The scripts work without Codex or any AI service. Arduino IDE is optional. The workflow below follows the current setup and launcher tools; remaining fresh-machine and hardware qualification limits are listed separately.

Both hosts use the same desktop command names: `desktop-rebuild`, `desktop-local` and `desktop-card`, with `.sh` on Linux and `.cmd` on Windows. Choose the host-specific setup below. Hardware build/upload uses Linux tooling; native macOS launchers are unimplemented and WSL launch paths remain unvalidated. Browser use on macOS is a separate route with limited user observations.

For the experimental browser build, follow the separate tools-web setup and launch instructions. Its common `setup.sh`, `desktop-rebuild.sh` and `desktop-local.sh` build/launch one Wasm runtime; target selection happens in the page. The native board-folder workflow below does not apply to that browser route.

1. Getting started

Clone

You need Git and repository access before running its setup script. From the directory where you keep projects, in a Linux shell or Windows Command Prompt with Git available:

```
git clone https://github.com/apws/261007-RunCPM-7xtgv-terminal.git
cd 261007-RunCPM-7xtgv-terminal
```

On Windows, a writable local directory such as `C:\work` is suitable; use `cd /d C:\work` in Command Prompt before cloning. If Git is not installed, extract a repository source ZIP instead, or install MSYS2 and Git first. For the latter route, complete the MSYS2 update and run `pacman -S --needed git` in UCRT64, then use the clone commands there.

- Use a new destination if that directory already exists; do not overwrite a checkout.
- A clone includes committed `cpmcard/` files, not another machine's uncommitted settings or physical SD card.
- RunCPM-derived sources are already included. Do not clone upstream RunCPM over them; no RunCPM submodule step is needed.
- Generated binaries are not included: build before the first launch.

Run explicit setup

Linux prerequisites and setup

From the repository root, as your **normal user**, not through sudo:

```
./src/RunCPM_7xtgv-terminal/tools/setup.sh --dry-run
./src/RunCPM_7xtgv-terminal/tools/setup.sh
```

The first command only previews; the second asks before installing.

- Detects Arch-based, Debian-based and Fedora-based distributions.
- Installs system tools, desktop C++/CMake/SDL2 support, Arduino CLI, and pinned cores/libraries for all six boards. There is no desktop-only setup mode.
- Uses sudo for system packages, but installs Arduino components as your user. If sudo is absent, have an administrator install/configure it first.
- Does not upgrade the whole system, build, flash or populate a card.

- Offers USB serial-group access separately; desktop-only use does not need it.
- Konsole and Kitty are optional and are not installed by setup. With neither, the mirror uses your current terminal through tmux.

Arch-family use has been exercised. Debian/Fedora package selection has offline tests, but real fresh-machine installation remains unverified. The complete clean-clone gate is still pending; see Remaining fresh-machine acceptance.

Windows prerequisites and setup

a. Install x64 MSYS2 on Windows, for example in `C:\msys64`. Existing installations can be reused; reinstalling is not a normal setup step. No WSL, Linux VM or Arduino IDE is required.

b. Open **UCRT64** from that installation, for example `C:\msys64\ucrt64.exe`. Close any running project desktop/mirror sessions before updating. Run:

```
pacman -Syu
```

If pacman asks to close MSYS2 processes, accept, reopen UCRT64 and run the command again to finish the remaining updates. `-Suy` is equivalent to `-Syu`. Complete the full update before installing our packages; see the MSYS2 update instructions.

c. From your checkout, run `src\RunCPM_7xtgv-terminal\tools-win\setup.cmd` in **Windows Command Prompt** or double-click it in Explorer. For example, with a clone in `C:\work\261007-RunCPM-7xtgv-terminal`:

```
cd /d C:\work\261007-RunCPM-7xtgv-terminal\src\RunCPM_7xtgv-terminal\tools-win
setup.cmd --dry-run
setup.cmd
```

The preview does not install anything. Setup then asks before installing packages into the selected MSYS2. Check the printed `Using MSYS2:` directory matches the installation updated in step b. Common C: and D: locations are discovered automatically; for a custom location set `set "XTGV_MSYS2_ROOT=D:\apps\msys64"` in the same Command Prompt first.

d. Wait for setup to finish successfully, then choose a target folder and rebuild/launch as described below. Setup itself does neither.

Setup installs Git, curl, CA certificates, MSYS Python, tmux, Mintty and native UCRT64 GCC/CMake/Ninja/SDL2, together with their package dependencies. These stay in the selected MSYS2 installation. Checksum-verified Arduino_GFX 1.6.4 sources go under `build/RunCPM_7xtgv-terminal/desktop-win/_deps/` in the checkout. It does not add a global Windows PATH entry, install services/drivers or MCU cores, or write to the card. Rebuild creates the EXE and supporting DLLs under `build/`; launch uses those artifacts.

MSYS2 Bash and Windows Command Prompt use different syntax: run `pacman` in UCRT64, and the `.cmd` examples and `set "NAME=value"` in Command Prompt. The launchers start their own MSYS2 environment. A VirtualBox shared checkout also works, but local guest storage avoids shared-folder overhead for card I/O. See the Windows tooling notes for dependency troubleshooting, logs and diagnostic commands.

Open a board folder

Relative to the repository root, choose your host's launcher parent:

- Linux: `src/RunCPM_7xtgv-terminal/tools/targets/`.
- Windows: `src/RunCPM_7xtgv-terminal/tools-win/targets/`.

Both have the following folders. Choose a display ID, even for desktop use:

Vendor	Display ID	Folder under that path
WAVESHARE	`EC6_147`	`EC6_147 -- esp32c6-lcd-1-47`
GUITION	`EP4_10`	`EP4_10 -- jc8012p4a1c`
GUITION	`ES3_35`	`ES3_35 -- jc8048w535`
GUITION	`ES3_5`	`ES3_5 -- jc8048w550`
WAVESHARE	`RP2_147`	`RP2_147 -- rp2350-lcd-1-47-a`
WAVESHARE	`RP2_2`	`RP2_2 -- rp2350-touch-lcd-2`

For example, from the repository root on Linux:

```
cd "src/RunCPM_7xtgv-terminal/tools/targets/EP4_10 -- jc8012p4a1c"
```

Each shortcut already selects its board; no `terminal.conf` edit is needed to choose the target for these shortcuts.

Linux	Windows	Action
`desktop-rebuild.sh`	`desktop-rebuild.cmd`	Build this board's native desktop executable; no MCU access.
`desktop-local.sh`	`desktop-local.cmd`	Launch its last build with repository `cpmcard/` and a mirror.
`desktop-card.sh`	`desktop-card.cmd`	Launch its last build with a selected external card and a mirror.
`rebuild.sh`	—	Build and stage firmware for the physical board; no upload.
`reflash.sh`	—	Upload staged firmware and restore the mirror; no build.
`mirror.sh`	—	Attach to the physical board; no build or upload.

Run `.sh` scripts from a Linux terminal or `.cmd` files from Windows Command Prompt; `.cmd` files can also be double-clicked. Keep folder spaces quoted.

A file manager is optional: Midnight Commander, Double Commander or Krusader can make navigating target folders convenient. Use its terminal/run action so launcher output and prompts remain visible; keyboard shortcuts depend on the file manager's configuration.

Build, then launch

From the chosen board folder, execute these separately on Linux:

```
./desktop-rebuild.sh
./desktop-local.sh
```

On Windows, for example from the repository root in Command Prompt:

```
cd "src\RunCPM_7xtgv-terminal\tools-win\targets\ES3_35 -- jc8048w535"
desktop-rebuild.cmd
desktop-local.cmd
```

Linux opens Konsole/Kitty or the current terminal; Windows opens Mintty. The emulator's CP/M commands, INI configuration and desktop controls are shared. Native window management and host-terminal options can differ.

- Wait for the build to finish successfully before launching.
- The SDL window is the board-sized display; **type into the mirror**. The mirror shows the complete logical terminal, even when the panel is panned.

- `VTINFO` at the CP/M prompt reports the running module, build and geometry.
- After changing source, rebuild again. Launch never silently recompiles.



A normal desktop run is **read/write**. Guest applications and INI/history autosaves can modify versioned files in `cpmcard/`. For experiments, use an isolated checkout or prepared card copy; see 2. Desktop sessions and cards.

For everyday controls and examples, continue with the User Guide. For automation and headless tests, use the developer desktop guide.

2. Desktop sessions and cards

Repository working card

Run `desktop-local.sh` on Linux or `desktop-local.cmd` on Windows to use the checkout's `cpmcard/`. Its location is independent of the shell's current directory. Guest files, INI settings and history changes persist in that card.

For an isolated experiment, use a separate checkout or a prepared card copy. The card guide describes exporting the complete working card; stop writers before copying. Developer overrides and platform validation details are in TRG: desktop host tooling. No launch copies, synchronizes or normalizes card content automatically.

External SD/USB card

Run `desktop-card` with the mounted card's root directory containing `A/0`. The command name and option are the same on both hosts; use the appropriate extension and actual mount path or drive letter.

Linux:

```
./desktop-card.sh --card-root "/run/media/$USER/RunCPM"
```

Windows Command Prompt:

```
desktop-card.cmd --card-root "E:\"
```

These paths are examples; the launcher does not guess the reader. Both modes open the selected card read/write. Use uppercase guest filenames for compatibility with case-sensitive host filesystems. Close the session before unmounting or removing the card.

Session ownership and shutdown

- Do not run two writers against the same card tree. Local and physical cards are independent copies; there is no automatic synchronization.
- Close the owning mirror or SDL window, or use guest `EXIT`, to stop the desktop session. Wait for shutdown before unmounting/removing the card.
- Guest `REBOOT` restarts the desktop process and mirror; `REFLASH` cannot invoke a hardware uploader from a desktop session.
- Desktop mirrors use isolated sessions, separate from the 7xtgv-terminal hardware mirror session (`7xtgv-terminal`).

- SDL requests always-on-top but can be minimized. Clicking it or using <kbd>Alt</kbd><kbd>Tab</kbd> can take keyboard focus: reselect the mirror to type.
- SDL backlight is a compensated brightness simulation. It does not establish physical luminance, MCU speed, SPI bandwidth, panel timing or electrical behavior.

Before pulling updates into a used checkout, close its desktop session and inspect `git status --short`. Preserve wanted local card changes; do not reset them merely to make Git accept a pull.

3. Physical hardware

Modules and vendor references

These references identify the hardware families used by our targets. Match module suffixes, connectors and the actual board before buying or flashing; a manufacturer's touch, wireless or other feature is not automatically implemented by this terminal. Display dimensions in the project use its chosen orientation.

Display ID	Manufacturer module	Product and technical references
`EC6_147`	Waveshare ESP32-C6-LCD-1.47	Product; wiki. Non-Touch model.
`RP2_147`	Waveshare RP2350-LCD-1.47-A	Product family; wiki. Type-C A variant; Type-A B is different.
`RP2_2`	Waveshare RP2350-Touch-LCD-2	Product; wiki.
`ES3_35`	Guition JC3248W535 family	Manufacturer specification, JC3248W535C_I_Y. See the software-name note below.
`ES3_5`	Guition JC8048W550 family	Product, N_I variant; Manufacturer specification, N/R/C variants.
`EP4_10`	Guition JC8012P4A1C_I_W family	Supplier listing, I_W_Y1; Manufacturer specification.

Guition's download index is a fallback if a specification URL changes. The W535 specification identifies an ESP32-S3, 320x480 AXS15231B module, consistent with our 480x320 landscape target. We identify JC3248W535 as the probable original module name; the precise physical suffix still needs confirmation. The existing software target, folder and boot identity remain `jc8048w535`, with display ID `ES3_35`; use that spelling in commands. This documentation correction does not rename or requalify hardware.

Build and upload

Choose the folder for the **connected physical module**. From that folder:

```
./rebuild.sh
./reflash.sh
```

- Rebuild needs no device. It stages a fresh candidate under that target's `new/`.

- Reflash uploads existing `new/`, falling back to `current/` only if `new/` is absent. An old `new/` is still old firmware: check the embedded build stamp.
- Upload stops the shared hardware mirror, restarts CP/M and loses running application state. The workflow attempts to restore the visible mirror afterward.
- Neither command promotes the candidate to `current/` or changes `terminal.conf`.
- Confirm boot `Build`, module identity, storage and the CP/M prompt; use `VTINFO` if needed. Upload success alone is not a hardware test.

Connection prompts and target guards

Family	What to do
ESP32	Normally upload over the running serial connection. If no serial device is found, the script asks you to connect the named module and press Enter.
RP2350	Connect in BOOTSEL mode. One eligible BOOTSEL disk already present skips the prompt; otherwise the script asks you to connect it in BOOTSEL and press Enter.

- ESP32 firmware identity mismatches block upload. Resolve board/target selection; never bypass a known mismatch.
- Only intentional first installation or recovery of unidentified ESP firmware uses `--allow-unidentified-target TARGET`, with the exact long target ID. Normal launchers do not supply it automatically.
- RP2350 checks the chip family and staged UF2 target. The ROM cannot identify which display is attached: verify the physical module yourself. More than one eligible disk needs an explicit `--chip-id HEX16` selector.
- EC6_147 does not normally need BOOTSEL. If upload reset leaves its panel blank, press hardware RESET; this recovery note is also in the upload log.

For ambiguous serial selection, specify the actual device, for example `./reflash.sh --port /dev/serial/by-id/YOUR-DEVICE`. Do not assume `/dev/ttyUSB0` identifies the board: some target defaults use generic port names. Never open two readers on the same serial port.

Mirror without reflashing

From the same board folder:

```
./mirror.sh
```

This attaches without rebuilding, uploading or deliberately resetting CP/M. Unlike reflash's missing-device prompt, mirror-only discovery can stop with an error: connect the module or specify `--port DEVICE`, then retry. The shared 7xtgv-terminal hardware mirror session is named `7xtgv-terminal`; do not reuse a stale session from a different board. Ending a hardware mirror does not itself stop CP/M; ending the owning **desktop** mirror stops that emulator.

USB permissions

From the repository root, with the device connected:

```
./src/RunCPM_7xtgv-terminal/tools/setup.sh --serial-only
```

This skips dependency installation and offers device-group access separately. Use a fresh login after group changes so all applications see the membership. The mirror can enter already-configured group membership through `newgrp`, or request a temporary user ACL when needed. An ACL may disappear when USB recreates the device node. None of this is needed for desktop PTYs.

Firmware is not the card

Flashing does not install CP/M programs or copy `cpmcard/` onto an SD card. Use RunCPM's FAT layout: drive A/user 0 is `A/0`; user directories are hexadecimal `0`--`F`, under drives `A`--`P`. See the working-card guide before preparing a card. Do not overwrite a working card's INI/history with an older repository copy.

VTPUT/VTGET transfer requires the helpers already on the card; it is not a blank-card bootstrap. Building firmware does not require reassembling guest COM files. Changed guest sources have separate build/install steps and assembler dialects; see the Reference Guide.

4. Build reference and troubleshooting

Quiet output, logs and optional switches

- Rebuild/reflash and desktop shortcuts show concise steps. Full compiler, uploader, image/card paths and section delimiters are in same-named `.log` files beside those scripts, replaced each run. Save a log before rerunning if you need earlier evidence. `mirror.sh` does not create a matching log.
- Failures report a short reason. If upload was blocked, the existing firmware remains; a newly opened mirror is not proof that new firmware was flashed.
- Rebuild/reflash and desktop shortcuts accept `--dry-run` for a read-only workflow preview. Desktop launch previews still validate the chosen card path. `mirror.sh` has no dry-run option; use `--help` without opening a connection.
- Hardware `--verbose` exposes live compiler/uploader output. Desktop shortcuts stay quiet; use their log or the developer CLI for diagnostics.
- Desktop rebuilds on Linux and Windows show build-step percentages from Make or Ninja while compiler details stay in the log. A terminal updates one line; redirected output keeps occasional progress lines. Percentages measure build work items, not remaining time; configuration and DLL staging are separate steps.
- Reflash, mirror and desktop launch accept `--mirror terminal`, `--mirror konsole` or `--mirror kitty`. Automatic selection prefers Konsole, then Kitty when a GUI session is available, otherwise the current terminal.
- Missing tools are reported, not installed behind your back. Run explicit setup for dependencies; investigate source errors separately.

Optional optimization selection

Defaults are the starting point: desktop uses `-O2` with debug symbols; hardware retains each profile's measured/default setting. To experiment, from a board folder:

```
./desktop-rebuild.sh -O2
# Separate hardware build, only if wanted:
./rebuild.sh -Os
```

- Accepted levels: `-O0`, `-Og`, `-O1`, `-O2`, `-O3`, `-Os`. The longer forms `--optimization O2` and `--optimization default` also work.
- Selection applies only to that build invocation. Omission restores defaults; reflash and launch use the existing artifact and reject optimization options.
- Hardware overrides affect compiled sources, not precompiled SDK archives. Unoptimized/debug firmware can exceed MCU memory; O3 is not always faster.

- Benchmark correctness and speed before adopting a hardware override.

Prerequisites and manual setup details

Use the appropriate host setup above first. This **Linux** inventory is for diagnosing missing components or preparing a machine manually, not an extra sequence after setup.

Use	Requirements	Source/check
Common host tools	Linux, bash, Git, Python 3; use Python 3.10+ for desktop	No project pip environment needed by basic build/mirror tools.
Desktop compiler	C+17 compiler, CMake 3.19, build tool (setup installs make), pthreads	SDL2 development headers/library or compatible SDL2 package.
Desktop drawing	GFX Library for Arduino 1.6.4	Default `$HOME/Arduino/libraries/GFX_Library_for_Arduino`; override build with `--gfx /path/to/library` if your sketchbook differs.
Firmware compiler	Arduino CLI	`ARDUINO_CLI`, PATH, or `$HOME/.local/bin/arduino-cli`; board packages provide cross-compilers/SDKs.
ESP32 firmware	Arduino-ESP32 3.1.0	All four ESP32 profiles pin this version and package index in `sketch.yaml`.
RP2350 firmware	Arduino-Pico 5.4.4	Both RP2350 profiles pin this version and package index.
Firmware libraries	GFX Library for Arduino 1.6.4; SdFat 2.3.0	Pinned in each profile; global installation alone does not prove profile resolution.
Mirror	tmux and an interactive terminal	Konsole/Kitty optional; desktop display additionally needs a graphical session.
Hardware upload	USB data cable, serial permissions; board-package tools	RP2350 also uses findmnt/udisksctl for BOOTSEL discovery/mounting.
Guest/helper development	Appropriate 8080/Z80 assembler or BASIC environment	Not needed to use supplied COM files or compile firmware.
Manual rendering	AsciiDoc renderer or browser extension	Optional; see repository README for browser viewing.

Native desktop builds do not use MCU cores, SdFat hardware drivers or USB upload tools, even though combined Linux setup installs them. They compile the real GFX library plus the project's host storage adapter.

Setup behavior and versions

- `setup.sh` uses `/etc/os-release`: exact `ID`, then ordered `ID_LIKE` families. Arch uses pacman; Debian/Ubuntu uses apt-get; Fedora uses dnf. Unsupported families stop before installation.
- Debian refreshes package indexes; no branch performs a full system upgrade.
- Existing Arduino CLI is reused. If absent, setup downloads checksummed CLI 1.5.0 for Linux x86_64 or aarch64 into `$HOME/.local/bin`.
- The 261004 working-machine observation was CLI `1.5.2-rc.1` (`fef6e48df`), not a tested minimum or a reason to require a prerelease.
- `XTGV_ARDUINO_CONFIG` selects an explicit Arduino configuration file. A custom sketchbook may require desktop `--gfx` as above.
- `--yes` approves dependency setup, not USB group changes. With it, run `--serial-only` separately if hardware access is needed.
- `--with-desktop` remains a compatibility alias: desktop packages are now included by default.

Allow network access and several GB of working space. There is no measured minimum disk/RAM requirement or universal first-build time estimate. The first profile build may download isolated dependencies even after setup. Use pinned versions before diagnosing a newer core's API/ABI differences.

For manual installation, the exact package list, indexes and commands are in setup.sh; versions/board options are in sketch.yaml. Do not force-overwrite an existing Arduino configuration. Useful read-only checks are `arduino-cli version`, `arduino-cli config dump`, `arduino-cli core list` and `arduino-cli lib list`. No separate hand-installed ESP-IDF or Pico SDK is part of the normal build path.

Artifacts and low-level build entry points

All paths below are relative to the repository root:

Path	Meaning
`build/RunCPM_7xtgv-terminal/TARGET/new/`	Successfully staged MCU candidate; `.ino.bin` for ESP32, `.ino.uf2` for RP2350.
`build/RunCPM_7xtgv-terminal/TARGET/current/`	Separately promoted hardware-accepted image, if one exists.
`build/RunCPM_7xtgv-terminal/desktop/TARGET/xtgv_desktop`	Linux native executable for that board; never an MCU upload image.
`build/RunCPM_7xtgv-terminal/desktop-win/msys2-ucrt64/TARGET/xtgv_desktop.exe`	Windows native executable, with supporting DLLs alongside it. Use the source-folder launchers to run it.

Here `TARGET` is the long ID after `--` in the folder table, without spaces. Successful rebuild leaves hardware `current/` untouched. Promotion and archive recovery are separate developer operations; some low-level recovery options also upload, so do not treat them as harmless file management.

The hardware shortcut wraps this command, from the repository root:

```
python3 src/RunCPM_7xtgv-terminal/tools/reflash.py --target jc8012p4a1c --build-only
```

The wrapper reads `tools/targets/*.json` and `sketch.yaml`, preserves platform flags and supplies target/product/SPI flags plus a fresh `YYMMDD-HHMM` stamp. A bare `arduino-cli compile --profile …` omits wrapper-supplied metadata and is not the supported build recipe. Do not replace the bundled product fork with upstream HEAD, mix target binaries, or build as root.

For upload, prefer the target `reflash.sh`: it manages the mirror and RP2350 BOOTSEL path. Bare `tools/reflash.py` upload is not an RP2350 recipe. Developer tests and direct mirror entry points are in the Technical Reference Guide.

Troubleshooting

Symptom	Check/remedy
No executable / no staged firmware	Run the matching desktop or hardware rebuild first; clone does not contain build artifacts.
Old build still running	Rebuild is separate from launch/reflash. Check embedded stamp and running `VTINFO`; a blocked upload did not install the candidate.
Missing compiler, CMake, SDL2 or Arduino CLI	Run explicit setup; check PATH/`ARDUINO_CLI`. For GFX, verify version/path or use build `--gfx`.
Windows setup reports conflicting package versions	Complete `pacman -Syu` in the same MSYS2 installation printed by setup. Reopen and repeat after core updates. If it still fails, keep the full update output and check the root with `cygpath -w /`; do not force dependency checks or remove compiler libraries.
Core/library download failure	Check network/proxy/CA certificates and disk space; keep pinned versions. Do not change HAL code to fix a download.
Mounted-card path rejected	Mount the card first and pass its root containing `A/0` to desktop-card. Developer directory overrides are described in TRG.
SDL ignores typing	Focus the mirror. SDL is output-only even when its title bar or Alt Tab gives it focus.
SDL too large / too dark	Choose a smaller board display or adjust `--window-scale 1` through `9` at launch. The SDL window is fixed-size; scale does not alter guest geometry. Use terminal backlight controls for brightness.
Missing serial device / permission denied	Check cable, actual port, permissions and competing readers. Use setup's `--serial-only` for group access.
Wrong board or multiple candidates	Resolve selection explicitly. Do not bypass a firmware mismatch or guess among BOOTSEL disks.
No graphical mirror launcher	Use `--mirror terminal` with tmux installed. Desktop SDL still needs a graphical session.
SD failure / CPU halted	Check physical card format/layout, connection and power. Upload does not provision guest files.
Mirror not restored after upload	Check device re-enumeration/reset, then use this board's `mirror.sh` with the correct port.

Remaining fresh-machine acceptance

Existing-machine builds, offline installer tests and desktop rendering do **not** complete this gate. Still required:

1. Record a published commit and Linux/Python/CLI versions in a disposable clone.
2. Test declared setup on the real distro with empty Arduino data/download/user directories and build caches, without private ZIPs or modified local libraries.
3. Build all six hardware targets and desktop selections; retain dependency logs, effective flags, sizes and remediation steps.
4. Confirm target identity and new stamp in artifacts; record download/disk needs.
5. Separately check GUI/mirror lifecycle, USB permissions, physical upload and initial card preparation. Compiler-only success cannot establish those.

Offline launcher tests use fake installers/temporary fixtures. Native tests are described in the Technical Reference Guide; skipped dependency tests are not completed qualification. Physical panel/backlight/SPI/electrical acceptance remains separate from desktop checks.

Dependency-isolation helper

For an intentional download/build test, from the repository root:

```
# Published checkout and all hardware targets; substitute the desired commit:
./src/RunCPM_7xtgv-terminal/tools/fresh-build.sh --target all --ref COMMIT

# Alternative: current non-ignored working files, NOT published-clone proof:
./src/RunCPM_7xtgv-terminal/tools/fresh-build.sh --snapshot --target esp32c6-lcd-1-47
```

- Downloads checksummed Arduino CLI, installs baseline cores/libraries in empty temporary directories and runs the normal hardware build-only wrapper.
- Uses one explicit `XTGV_ARDUINO_CONFIG` for property resolution and compilation.
- `--use-cli PATH` reuses only the executable, not its installed packages/cache; `--source URL_OR_PATH` changes clone source. See `--help` for all options.
- Logs/artifacts remain in a new temporary directory. No flash, sudo, device access or automatic removal of that directory.
- Requires Linux, bash/coreutils, Git, Python, curl, tar, CA certificates and network access already present. It does not install missing OS tools, build SDL desktop or simulate an entirely fresh operating system.

A separate machine/VM/container check is still needed. Record test evidence, not just the existence of this helper, before claiming reproducibility.